

Assignment 3 - Cache-Friendly Graph Processing

Rami Jay Ketanbhai (2024CS10510)

Rohit Shakya (2024CS10410)

April 2026

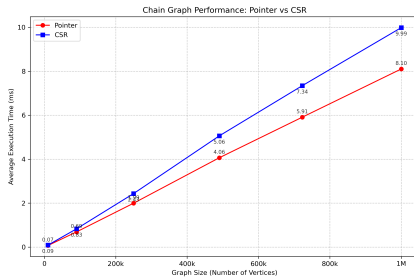
1 Question 1: Runtime Comparison

Based on our benchmarks, the **CSR** implementation is the superior choice for structured and dense graphs, while the **Pointer** implementation holds a very small advantage in chain topologies.

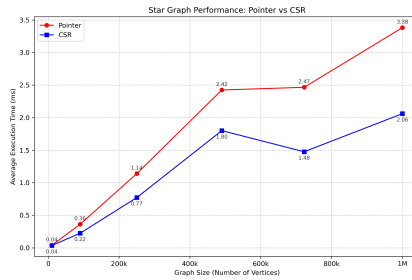
Also please note that the overhead for **CSR** graph has not been subtracted and increasing iterations in general is enough to prevent the conversion overhead to play any role in results.

Topology	Size (N)	Ptr Time (ms)	CSR Time (ms)	Improvement
Chain	90k	0.69	0.83	0.83x
Chain	490k	4.06	5.06	0.80x
Chain	1M	8.10	9.99	0.81x
Star	90k	0.36	0.22	1.64x
Star	490k	2.42	1.80	1.34x
Star	1M	3.38	2.06	1.64x
Grid	90k	0.94	0.36	2.61x
Grid	490k	8.16	2.82	2.89x
Grid	1M	16.90	6.54	2.58x

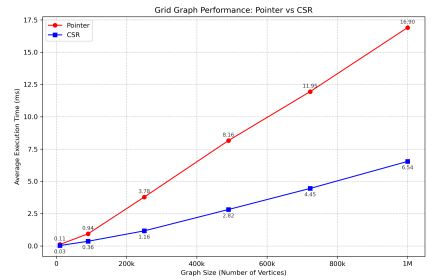
Table 1: Runtime improvement across basic graph topologies at varying sizes.



(a) Chain Topology



(b) Star Topology



(c) Grid Topology

Figure 1: Runtime comparison for basic graph topologies.

Graph Explanations (Basic Topologies)

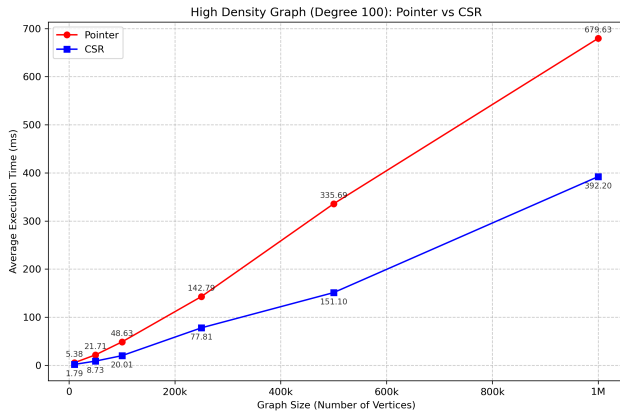
- **Chain Graphs:** The Pointer implementation is consistently faster (e.g., 8.10ms vs 9.99ms). **Why?** In a chain, every vertex has exactly one neighbor (degree = 1). CSR must read from `row_ptr[u]` and `row_ptr[u+1]` just to find the boundaries, and then read one integer from `col_idx`. This

double-array lookup provides no spatial locality benefit for a single neighbor, making the simple `ptr->next` dereference faster.

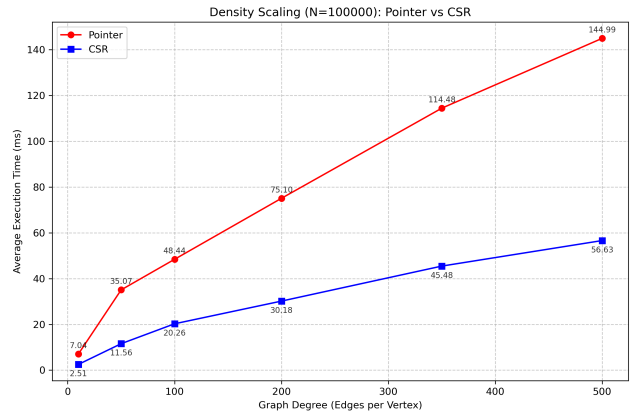
- **Star and Grid Graphs:** CSR takes a commanding lead, achieving a **2.58x speedup** on the 1M vertex Grid graph. **Why?** As soon as nodes have multiple neighbors, CSR’s contiguous memory layout allows the CPU to fetch multiple neighbors into the L1 cache simultaneously, completely bypassing the memory stall penalties that the Pointer implementation pays to traverse linked lists.

ER Density (N=100k)	Ptr Time (ms)	CSR Time (ms)	Improvement
10	7.04	2.51	2.80x
100	48.44	20.26	2.39x
500	144.99	56.63	2.56x

Table 2: Runtime improvement scaling by Graph Density.



(a) ER Scaling by Size (N)



(b) ER Scaling by Density (deg)

Figure 2: Performance scaling on random Erdős–Rényi graphs.

Graph Explanations (Erdős–Rényi Scaling)

As shown in the ER plots, CSR scales remarkably better with both size and density. At $N = 100k$ and $deg = 500$, CSR is **2.56x faster** (56.63ms vs 144.99ms). **Why?** High-density graphs require iterating over massive neighbor lists. The Pointer approach forces the CPU to constantly chase random heap allocations, destroying pipeline efficiency. CSR streams these massive neighbor lists contiguously, leveraging hardware prefetchers to mask RAM latency entirely.

2 Question 2: Cache Behavior

We used *Cachegrind* to measure the algorithmic memory efficiency of the traversal algorithms.

Degree	Ptr D1	CSR D1	D1 Imprv.	Ptr LLd	CSR LLd	LLd Imprv.
8	112,480	69,488	1.62x	50,995	7,143	7.14x
16	196,693	105,489	1.86x	98,449	13,968	7.05x
50	553,507	245,810	2.25x	284,450	75,305	3.78x
100	1,085,410	450,552	2.41x	555,375	150,969	3.68x

Table 3: Misses and improvement across varying graph densities.

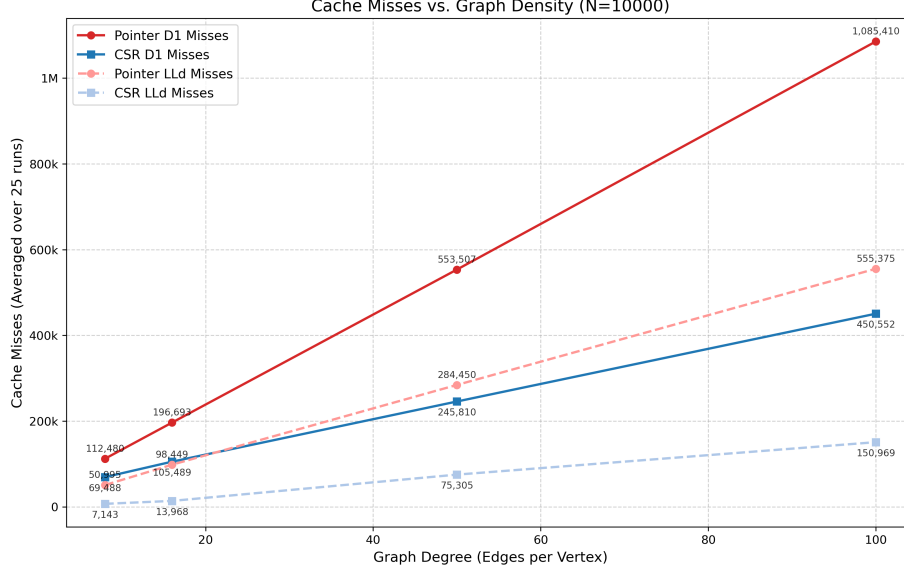


Figure 3: D1 and LLd misses vs. Graph Degree (averaged over 25 runs).

Graph Explanation (Cache Misses by Degree)

- D1 Misses:** At degree 100, Pointer suffers **1,085,410** misses while CSR generates only **450,552**. **Why?** In the pointer-based graph, each edge is stored in a dynamically allocated struct containing a destination vertex (4 bytes) and a next pointer (8 bytes), typically aligning to 16 bytes. CSR stores neighbors in a dense 4-byte array. Because the CSR layout is 4x more compact, a single L1 cache line fetch pulls in four times as many neighbors, massively reducing D1 misses.
- LLd Misses:** At degree 100, Pointer suffers **555,375** misses while CSR generates only **150,969** (a $\sim 3.68x$ difference). **Why is the gap so large?** For $N = 10,000$, the `dist[]` array is only 40 KB, which fits entirely inside the simulated 1 MB Last-Level Cache. Thus, LLd misses are driven entirely by graph structure capacity. At degree 100, the CSR edge array is roughly 4 MB, while the Pointer structure is roughly 16 MB. The massive 16 MB footprint thrashes the 1 MB cache, causing severe capacity misses that the compact CSR representation mitigates.

3 Question 3: Memory Layout Explanation

CSR tends to perform better than pointer graphs due to three key architectural factors:

- Spatial Locality:** CSR stores a vertex’s neighbors in a contiguous block in the `col_idx` array. When the CPU fetches one neighbor, the entire cache line is filled with the *next* several neighbors,

making them functionally "free" to access.

2. **Contiguous Arrays:** Unlike linked lists which require `next` pointers and heap metadata, CSR is 100% payload. This drastically increases the "effective cache capacity."
3. **Reduced Pointer Chasing:** In a pointer graph, every edge traversal requires a dereference (`e = e->next`). If memory is fragmented (as is common in real-world scenarios), the CPU stalls waiting for RAM at every step. CSR uses simple integer incrementing, which is effortlessly handled by the CPU's branch predictor and prefetcher.

4 Question 4: Varying Cache Parameters

We simulated hardware variations to observe how each representation reacts to different cache constraints. (Note: Iterations were increased to amortize CSR conversion overhead).

4.1 1. How does increasing cache size affect BFS performance?

Cache Size (B)	Ptr D1	CSR D1	D1 Imprv.
8192	139,436	105,582	1.32x
32768	112,480	69,487	1.62x
131072	67,226	24,977	2.69x

Table 4: D1 Misses and improvement across cache sizes.

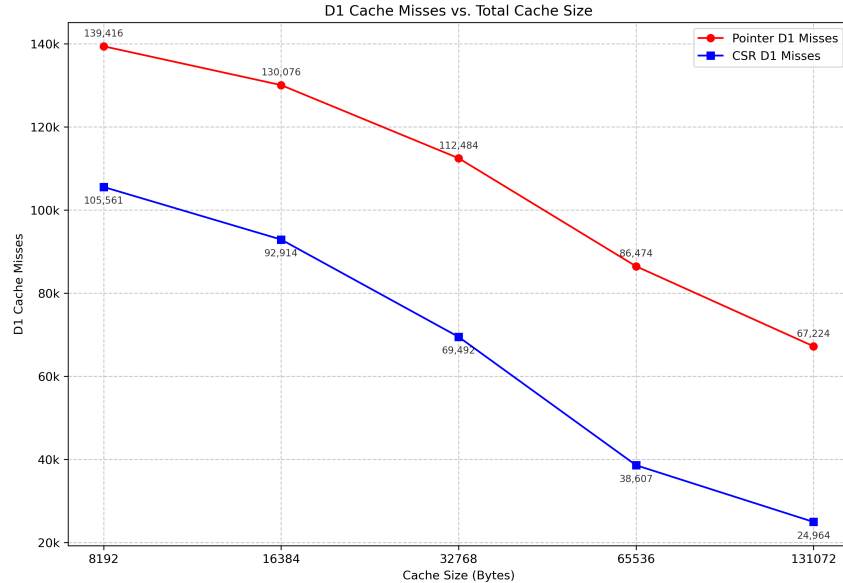


Figure 4: D1 and LLd misses vs. Total Cache Size.

Answer & Graph Explanation: Increasing the cache size improves BFS performance by directly reducing **capacity misses** for both implementations. However, as seen in the graph, CSR's curve drops much faster, widening the performance gap to 2.69x at 128KB. **Why?** Because CSR's memory footprint

is exponentially smaller, a 128KB cache can hold a significant, meaningful portion of the CSR graph’s active working set. The same 128KB cache barely scratches the surface of the bloated 16-byte-per-edge Pointer structure.

4.2 2. How does increasing associativity affect the results?

Associativity	Ptr D1	CSR D1	D1 Imprv.
1	131,196	87,957	1.49x
2	112,095	69,164	1.62x
8	112,480	69,487	1.62x

Table 5: D1 Misses and improvement across cache associativities.

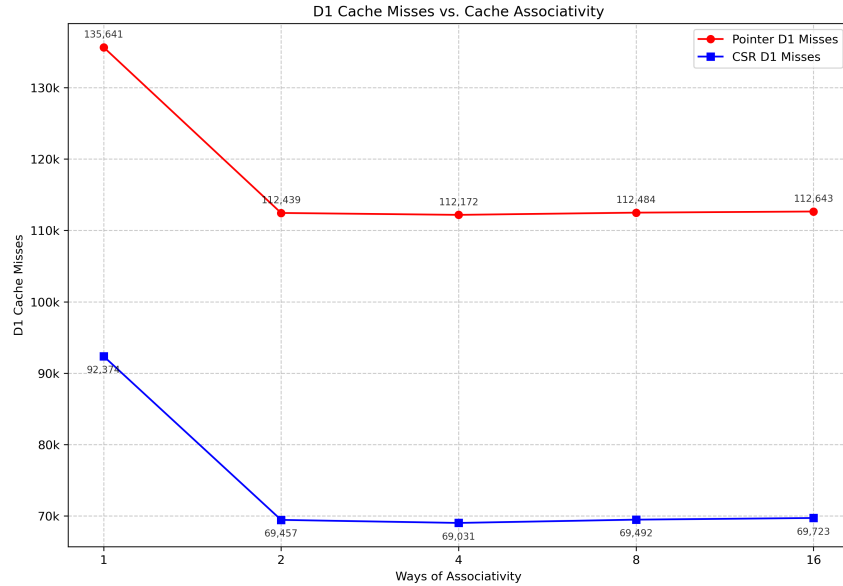


Figure 5: D1 and LLd misses vs. Cache Associativity.

Answer & Graph Explanation: Increasing associativity primarily reduces **conflict misses**. In the graph, the most dramatic performance gain occurs exactly when transitioning from a 1-way (direct-mapped) to a 2-way associative cache (e.g., 131k $\rightarrow$ 112k for Pointer D1). **Why does the graph go flat after 2-way?** Once the cache has enough "ways" to prevent overlapping graph and `dist[]` array memory addresses from constantly evicting each other, adding more associativity provides no benefit. The remaining misses on the flat line are purely capacity and compulsory misses dictated by the size of the graph.

4.3 3. How does changing line size influence performance?

Line Size (B)	Ptr D1	CSR D1	D1 Imprv.
64	112,478	69,484	1.62x
128	84,109	60,868	1.38x
512	65,932	59,546	1.11x

Table 6: D1 Misses and improvement across cache line sizes.

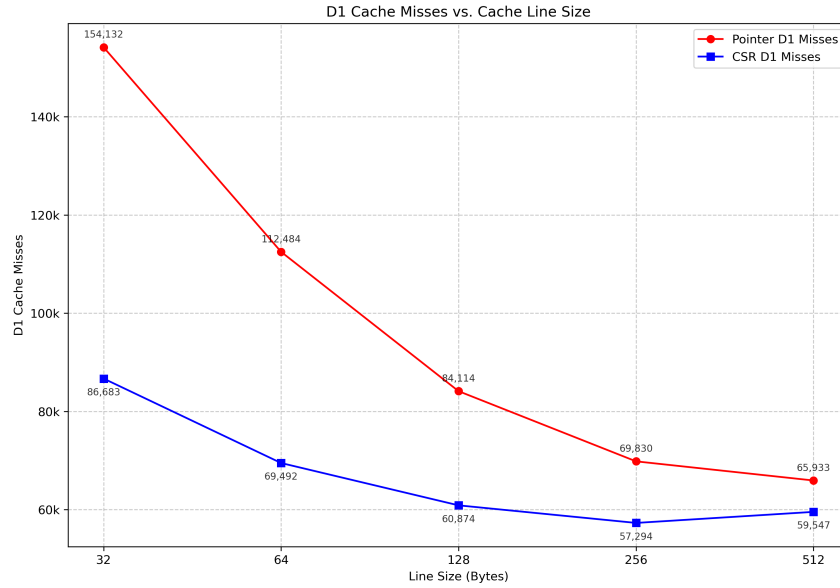


Figure 6: D1 and L1d misses vs. Cache Line Size.

Answer & Graph Explanation: Changing the cache line size determines how much adjacent memory is pulled into the cache during a single fetch. As the line size grows, the number of misses drops for both because fewer individual RAM fetches are required to scan a neighborhood. However, doing so blindly can lead to cache pollution if the adjacent data isn't actually useful.

4.4 4. Does CSR benefit more from larger cache lines than pointer graphs? Why?

Answer: Yes, CSR benefits **massively** more from larger cache lines.

If you look at the L1d dashed lines in the line-size graph, the CSR L1d misses drop from 7,144 at 64B to just 905 at 512B—a near-total elimination of compulsory misses.

Why? A cache line fetch is only beneficial if the data pulled in is useful (Spatial Locality).

- **CSR:** Because neighbors are stored as a packed array of 4-byte integers, a 512B cache line fetch pulls in exactly **128 useful neighbors** at once.
- **Pointer:** The **Edge** struct is 16 bytes, and memory allocator padding can push it higher. Fetching a 512B cache line for a Pointer graph might pull in 1 useful edge and 496 bytes of completely useless heap metadata. This forces the hardware to waste bandwidth transferring junk data, which causes the diminishing returns seen in the Pointer curve.